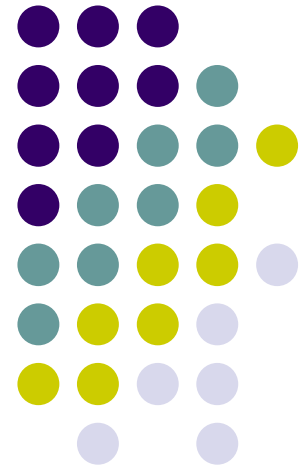


Unidad VII

Pruebas



Pruebas

Conjunto de actividades que se planean con anticipación y se realizan de manera sistemática.

Se interpreta que el software ha fallado, cuando éste no hace lo que especifican los requerimientos.

- Razones de falla:
 - Especificación errónea u omisión de algún requerimiento.
 - Defecto en el diseño del sistema.
 - Defecto en el diseño del programa.
 - Implementación incorrecta o incompleta (código errado).

Pruebas

- Verificar y Validar

¿Estamos construyendo el producto correctamente?

¿Estamos construyendo el producto correcto?

- **El objetivo de la prueba**

Es el de demostrar la existencia de un defecto.

Luego, la prueba es destructiva, dado que la meta es descubrir defectos.

Una prueba se considera exitosa solamente cuando se descubre un defecto o cuando se produce una falla como resultado de los procedimientos de prueba.

Pruebas

Tipos de defectos en implementación

Defecto algorítmico:

- Bifurcar antes / después de tiempo.
- Probar para una condición errónea.
- Olvidar inicializar una variable o fijar constantes de un bucle.
- Olvidar la prueba para una condición particular.

Defecto de documentación:

La documentación no se corresponde con lo que el programa realmente hace.

Pruebas

Tipos de defectos en implementación

- Defecto por estrés o sobrecarga: ciertas estructuras se llenan hasta sobrepasar su capacidad especificada.
- Defecto de sincronización o de coordinación: el código que coordina eventos es inadecuado.
- Defecto de rendimiento o desempeño: el sistema no opera a la velocidad prescrita por los requerimientos.
- Defecto del hardware: no operan según lo documentado

Pruebas

Características generales

1. Para realizar pruebas efectivas un equipo de SW debe realizar RTF y deben ser efectivas.
2. Las pruebas comienzan a nivel de componentes y trabajan “hacia fuera”, hacia la integración de todo el sistema.
3. Diferentes técnicas de prueba son apropiadas en diferentes momentos.
4. La prueba la dirige el desarrollador de SW y en caso de proyectos grandes un grupo de pruebas independiente.
5. La prueba y la depuración son actividades independientes, pero la segunda siempre esta incluida.

Pruebas

Estrategias de Pruebas

- Prueba de Unidad – Código – Programador
- Prueba de integración – Diseño – Analista
 - Integración Ascendente
 - Integración Descendente
 - Regresión
- Prueba de validación – Requisitos – Cliente
 - Pruebas Alfa y Beta
- Pruebas de Sistema – Ingeniero de Sistemas
 - Prueba de recuperación
 - Prueba de seguridad
 - Prueba de resistencia
 - Prueba de desempeño

Pruebas

Fundamentos de las pruebas de SW

Facilidad de las pruebas.

Si es o no fácil de probar.

Operatividad.

«Cuanto mejor funcione, más eficientemente se puede probar.»

El sistema tiene pocos errores (los errores añaden sobrecarga de análisis y de generación de informes al proceso de prueba).

Ningún error bloquea la ejecución de las pruebas.

El producto evoluciona en fases funcionales (permite hacer en simultaneo el desarrollo y las pruebas).

Pruebas

Observabilidad.

«Lo que ves es lo que pruebas.»

Se genera una salida distinta para cada entrada.

Los estados y variables del sistema están visibles o se pueden consultar durante la ejecución.

Los estados y variables anteriores del sistema están visibles o se pueden consultar (por ejemplo, registros de transacción).

Todos los factores que afectan a los resultados están visibles.

Un resultado incorrecto se identifica fácilmente.

Se informa automáticamente de los errores internos.

El código fuente es accesible.

Controlabilidad.

«Cuanto mejor podamos controlar el software, más se puede automatizar y optimizar.»

El ingeniero de pruebas puede controlar directamente los estados y las variables del hardware y del software.

Pruebas

Capacidad de descomposición.

«Controlando el ámbito de las pruebas, podemos aislar más rápidamente los problemas y llevar a cabo mejores pruebas de regresión.»

El sistema software está construido con módulos independientes. Los módulos del software se pueden probar independientemente.

Simplicidad.

«Cuanto menos haya que probar, más rápidamente podremos probarlo.»

Simplicidad funcional (por ejemplo, el conjunto de características es el mínimo necesario para cumplir los requisitos).

Simplicidad estructural (por ejemplo, la arquitectura es modularizada para limitar la propagación de fallos).

Simplicidad del código (por ejemplo, se adopta un estándar de código para facilitar la inspección y el mantenimiento).

Pruebas

Estabilidad.

«Cuanto menos cambios, menos interrupciones a las pruebas.»

Los cambios del software son infrecuentes.

Los cambios del software están controlados.

Los cambios del software no invalidan las pruebas existentes.

El software se recupera bien de los fallos.

Facilidad de comprensión.

«Cuanta más información tengamos, más inteligentes serán las pruebas.»

El diseño se ha entendido perfectamente.

Las dependencias entre los componentes internos, externos y compartidos se han entendido perfectamente.

Se han comunicado los cambios del diseño.

La documentación técnica es accesible.

Pruebas

Características de la Prueba de SW

1. Una buena prueba tiene una alta probabilidad de encontrar un error. Para alcanzar este objetivo, el responsable de la prueba debe entender el software e intentar desarrollar una imagen mental de cómo podría fallar el software.
2. Una buena prueba no debe ser redundante. El tiempo y los recursos para las pruebas son limitados. No hay motivo para realizar una prueba que tiene el mismo propósito que otra.

Pruebas

Características de la Prueba de SW

3. Una buena prueba debería ser «la mejor de su clase» En un grupo de pruebas que tienen propósito similar, las limitaciones de tiempo y recursos pueden abogar por la ejecución de sólo un subconjunto de estas pruebas. En tales casos, se debería emplear la prueba que tenga la más alta probabilidad de descubrir una clase entera de errores.
4. Una buena prueba no debería ser ni demasiado sencilla ni demasiado compleja. Aunque es posible a veces combinar una serie de pruebas en un caso de prueba, los posibles efectos secundarios de estos fallos pueden enmascarar errores. En general, cada prueba debería realizarse separadamente.

Pruebas

Prueba de caja negra

Prueba de caja negra se refiere a las pruebas que se llevan a cabo sobre la interfaz del software.

Es decir, los casos de prueba pretenden demostrar que las funciones del software son operativas, que la entrada se acepta de forma adecuada y que se produce un resultado correcto, así como que la integridad de la información externa (por ejemplo, archivos de datos) se mantiene.

Una prueba de caja negra examina algunos aspectos del modelo fundamental del sistema sin tener mucho en cuenta la estructura lógica interna del software.

Pruebas

Prueba de caja negra

1. ¿Cómo se prueba la validez funcional?
2. ¿Cómo se prueba el rendimiento y el comportamiento del sistema?
3. ¿Qué clases de entrada compondrán unos buenos casos de prueba?
4. ¿Es el sistema particularmente sensible a ciertos valores de entrada?
5. ¿De qué forma están aislados los límites de una clase de datos?
6. ¿Qué volúmenes y niveles de datos tolerará el sistema?

Pruebas

Prueba de caja negra

¿Qué efectos sobre la operación del sistema tendrán combinaciones específicas de datos?

- Métodos gráficos de pruebas.
- Partición equivalente.
- Análisis de valores límites.
- Prueba de tabla ortogonal.

Pruebas

Prueba de caja blanca

La prueba de caja blanca del software se basa en el minucioso examen de los detalles procedimentales. Se comprueban los caminos lógicos del software proponiendo casos de prueba que ejerciten conjuntos específicos de condiciones y/o bucles.

Se puede examinar el «estado del programa» en varios puntos para determinar si el estado real coincide con el esperado o mencionado.

Las pruebas de caja blanca son diseñadas después de que exista un diseño de componente (o código fuente). El detalle de la lógica del programa debe estar disponible.

Pruebas

Prueba de caja blanca

La prueba de caja blanca, denominada a veces prueba de caja de cristal es un método de diseño de casos de prueba que usa la estructura de control del diseño procedimental para obtener los casos de prueba.

Mediante los métodos de prueba de caja blanca, el ingeniero del software puede obtener casos de prueba que

- (1) Garanticen que se ejercita por lo menos una vez todos los caminos independientes de cada módulo;
- (2) Ejerciten todas las decisiones lógicas en sus vertientes verdadera y falsa;
- (3) Ejecuten todos los bucles en sus límites y con sus límites operacionales;
- (4) Ejerciten las estructuras internas de datos para asegurar su validez.

Pruebas

Estrategias de pruebas orientadas a objetos

Pruebas de Unidad

Pruebas de integración

Prueba basadas en fallas

Casos de pruebas y jerarquías de clases

Prueba basada en escenarios

Estructura de superficie y de fondo en pruebas

Pruebas a nivel clases

Pruebas de comportamiento

Pruebas

Prueba de entornos especializados

- Prueba IGU

- Prueba Cliente servidor

Pruebas de funcionalidad de la aplicación

- Pruebas de servidor

- Prueba de BD

- Pruebas de transacción

- Pruebas de comunicación de red

Pruebas de documentación

Pruebas de Sistemas TR

- Pruebas de tareas

- Pruebas de comportamiento

- Pruebas de interfases

- Pruebas del sistema



FIN

Pruebas

